BÁO CÁO LAB 1 — MÃ HÓA ĐỐI XỨNG AES VỚI CRYPTO++

Môn học: An toàn thông tin / Mật mã ứng dụng **Đề tài:** Xây dựng công cụ dòng lệnh `aestool` hỗ trợ AES (ECB, CBC, OFB, CFB, CTR, XTS, CCM, GCM), chống lạm dụng, kiểm thử KAT & đo hiệu năng đa nền tảng

Thông tin	Nội dung
Họ tên / Nhóm	`[ĐIỀN: Họ tên – MSSV]`
Lớp / Giảng viên	`[ĐIỀN]`
Ngày thực hiện	12/11/2026
Mã nguồn	thư mục `lab1/` (CMake, C++17, Crypto++ 8.9.0)

Mục lục

1. Mục tiêu & phạm vi kỹ thuật
2. Cơ sở lý thuyết về AES và các chế độ hoạt động
3. Thiết kế công cụ CLI & cơ chế chống lạm dụng
4. Kiểm thử tính đúng đắn (KAT) & kiểm thử tiêu cực
5. Môi trường thực nghiệm
6. Phương pháp & kết quả đo hiệu năng
7. Thảo luận an toàn mật mã
8. Build đa nền tảng (CMake)
9. Phần mở rộng (Bonus): thư viện động + GUI
10. Hạn chế & hướng phát triển
11. Kết luận
12. Tài liệu tham khảo

- Phụ lục A – Academic Integrity & Ethics Statement
- Phụ lục B – Bảng tự chấm điểm (Self-grade Rubric)

> **Quy ước:** các mục đánh dấu `[ĐIỀN]` là thông tin phụ thuộc máy của bạn (CPU, RAM, phiên bản OS…) mà tôi không thể suy ra từ mã nguồn. Hãy thay trước khi nộp. Các kết quả chạy lệnh trong mục 3–4 do chính `aestool` sinh ra khi build từ mã nguồn trong gói `lab1.zip` (Ubuntu 24.04, GCC 13.3, Crypto++ 8.9.0).

1. Mục tiêu & phạm vi kỹ thuật

1.1 Mục tiêu học tập

Sau Lab 1, các mục tiêu đạt được gồm:

1. **Sử dụng thư viện Crypto++** (thư viện mật mã duy nhất được dùng; không dùng OpenSSL/libsodium) để cài đặt AES ở nhiều chế độ hoạt động qua một lớp bọc mỏng `aescore`.

2. **Quản lý IV/nonce an toàn:** kiểm tra độ dài chuẩn theo từng mode, tự sinh IV bằng `AutoSeededRandomPool` (CSPRNG), lưu IV vào header hoặc sidecar JSON.
3. **Chống lạm dụng mã hóa (misuse resistance):** cảnh báo/chặn ECB, từ chối nonce lặp (CTR/CCM/GCM), từ chối tham số sai, *fail closed* khi xác thực thất bại.
4. **Kiểm thử qua KATs của NIST** (SP 800-38A, GCM, SP 800-38C) và kiểm thử tiêu cực (wrong key, wrong IV, tamper, tag sai, IV sai độ dài).
5. **Đo hiệu năng thống kê chuẩn hóa:** warm-up, lặp lớn (N = 1000 » 30), báo cáo Mean / Median / SD / khoảng tin cậy 95%, 6 kích thước payload, so sánh Windows–Linux, stream–block, AEAD–non-AEAD.
6. **Build out-of-source bằng CMake** cho Windows và Linux, đóng gói lõi thành thư viện `.dll/.so` và gọi từ GUI Python (bonus).

1.2 Danh mục thuật toán & chế độ mã hóa

Công cụ hỗ trợ **AES-128/192/256** qua tham số `--mode`:

Mode	Loại	IV/Nonce	Tag	Ghi chú triển khai (Crypto++)
`ecb`	Block, không IV	–	–	PKCS#7; cảnh báo + giới hạn 16 KiB
`cbc`	Block	16 B	–	`CBC_Mode<AES>`, PKCS#7
`ofb`	Stream (từ block)	16 B	–	`OFB_Mode<AES>`
`cfb`	Stream (từ block)	16 B	–	`CFB_Mode<AES>` (CFB-128)
`ctr`	Stream (từ block)	16 B (nonce‖counter)	–	`CTR_Mode<AES>`
`xts`	Tweakable block	16 B (tweak)	–	`XTS_Mode<AES>`, khóa kép 32/64 B, hỗ trợ ciphertext stealing
`ccm`	**AEAD**	7–13 B (mặc định 12)	4–16 B (mặc định 16)	`CCM<AES, T>` — `T` là tham số biên dịch
`gcm`	**AEAD**	12 B	12–16 B (mặc định 16)	`GCM<AES>`

Độ dài khóa: 16/24/32 byte (AES-128/192/256); riêng XTS cần 32 hoặc 64 byte (hai khóa AES-128 hoặc AES-256). Trong các thử nghiệm của báo cáo ưu tiên **AES-256**.

1.3 Chế độ AEAD

- Cờ `--aead` là một **khẳng định tường minh**: nếu dùng với mode không phải CCM/GCM, chương trình từ chối ngay (`ERROR: --aead needs --mode ccm or gcm; 'cbc' has no authentication`, mã thoát 2).
- Dữ liệu xác thực phụ (AAD) truyền bằng `--aad FILE` hoặc `--aad-text STRING`; AAD được lưu trong header container và **được bao phủ bởi tag** nhưng không được mã hóa.
- Khi giải mã, tag được xác minh **trước khi** trả plaintext. Nếu tag sai, plaintext bị ghi đè bằng 0 trong bộ nhớ, ném `AuthError`, **không ghi file đầu ra** (fail closed), mã thoát `3`.

2. Cơ sở lý thuyết về AES và các chế độ hoạt động

2.1 AES — hoán vị khối 128 bit

AES (Rijndael, chuẩn FIPS-197) là **mã khối** xử lý khối 128 bit, khóa 128/192/256 bit, tương ứng **10/12/14 vòng**. Mỗi vòng gồm bốn phép biến đổi trên ma trận trạng thái 4×4 byte:

Bước	Tác dụng
`SubBytes`	Thay thế phi tuyến từng byte bằng S-box (nghịch đảo trong GF(2^8) + biến đổi affine) → tạo **confusion**
`ShiftRows`	Dịch vòng các hàng → khuếch tán giữa các cột
`MixColumns`	Nhân ma trận cố định trên GF(2^8) → khuếch tán trong cột (**diffusion**); bỏ qua ở vòng cuối
`AddRoundKey`	XOR với khóa vòng sinh từ *key schedule*

Bản thân AES chỉ là một **hoán vị** $E_K : \{0,1\}^{128} \to \{0,1\}^{128}$ cho mỗi khóa K. Để mã hóa thông điệp dài ta cần một **chế độ hoạt động (mode of operation)**; an toàn của hệ thống phụ thuộc vào mode và cách dùng IV/nonce **không kém** an toàn của AES.

Crypto++ tự chọn đường chạy tăng tốc phần cứng **AES-NI** (x86) hoặc ARMv8 Crypto khi CPU hỗ trợ; `aescore` có hàm `hardware_aes_available()` để kiểm tra. Điều này giải thích vì sao thông lượng đạt hàng trăm đến hàng nghìn MB/s (mục 6).

2.2 Các chế độ hoạt động — ký hiệu

- P_i, C_i: khối plaintext/ciphertext thứ i (128 bit); E_K, D_K: mã/giải mã một khối.
- IV: vector khởi tạo; $\oplus$: XOR; T: tag xác thực.

2.3 ECB — Electronic Codebook

$$C_i = E_K(P_i), \qquad P_i = D_K(C_i)$$

- Mỗi khối được mã độc lập → **cùng plaintext ⇒ cùng ciphertext** (tất định, không IV).
- Rò rỉ cấu trúc/mẫu dữ liệu (xem hình 7.1), cho phép tráo/cắt/ghép khối mà không bị phát hiện.
- **Không nên dùng** cho dữ liệu thực; chỉ phù hợp mã hóa đúng 1 khối ngẫu nhiên (ví dụ bọc khóa ngắn trong vài giao thức cũ). `aestool` cảnh báo và chặn dữ liệu > 16 KiB.

2.4 CBC — Cipher Block Chaining

$$C_0 = IV, \quad C_i = E_K(P_i \oplus C_{i-1}), \qquad P_i = D_K(C_i) \oplus C_{i-1}$$

- **Mã hóa tuần tự** (khối sau phụ thuộc khối trước) nhưng **giải mã song song được** → thông lượng giải mã cao hơn mã hóa (thể hiện rõ trong số đo, mục 6).
- IV phải **không thể đoán trước** (ngẫu nhiên) và độc lập mỗi lần mã hóa.
- Cần **padding** (PKCS#7) cho độ dài không chia hết 16 → sinh nguy cơ **padding oracle** nếu không có MAC.
- Không có tính toàn vẹn: sửa C_{i-1} làm thay đổi có kiểm soát các bit tương ứng trong P_i (và làm hỏng hoàn toàn P_{i-1}).

2.5 OFB — Output Feedback

$$O_0 = IV,\ O_i = E_K(O_{i-1}),\ C_i = P_i \oplus O_i$$

- Biến mã khối thành **mã dòng**: keystream O_i **độc lập với dữ liệu**, không cần padding.
- Keystream sinh tuần tự (không song song hóa cả mã lẫn giải mã), nhưng có thể tính trước.
- Dùng lại IV ⇒ trùng keystream (thảm họa như CTR).

2.5b CFB — Cipher Feedback

$$C_i = P_i \oplus E_K(C_{i-1}), \quad C_0 = IV$$

- Cũng là mã dòng, không padding; **giải mã song song được** (vì C_{i-1} đã biết) còn mã hóa tuần tự.
- Tự đồng bộ lại sau lỗi bit/mất khối (tính chất tự phục hồi).

2.6 CTR — Counter

$$C_i = P_i \oplus E_K(\text{Nonce} \,\|\, \text{Counter}_i)$$

- **Song song hóa hoàn toàn** cả mã và giải mã, truy cập ngẫu nhiên, không padding → là mode **nhanh nhất** trong các mode không AEAD (số đo: CTR 3137 MB/s ở 1 MB trên Linux).
- **Tuyệt đối không dùng lại cặp (Key, Nonce)**: nếu $C_1 = P_1 \oplus KS$ và $C_2 = P_2 \oplus KS$ thì $C_1 \oplus C_2 = P_1 \oplus P_2$ ⇒ rò rỉ plaintext (two-time pad).
- **Dễ bị biến đổi bit (malleable):** lật 1 bit ciphertext ⇒ lật đúng bit đó trong plaintext (mục 4.3).

2.7 XTS — XEX-based Tweaked-codebook with ciphertext Stealing

- Thiết kế cho **mã hóa đĩa/lưu trữ tĩnh** (IEEE 1619, NIST SP 800-38E). Dùng **hai khóa** K_1, K_2: tweak $T = E_{K_2}(\text{tweak}) \cdot \alpha^j$ trong GF(2^{128}), $C_j = E_{K_1}(P_j \oplus T) \oplus T$.
- Mỗi sector/đơn vị dữ liệu có tweak riêng ⇒ cùng nội dung ở sector khác nhau cho ciphertext khác nhau; hỗ trợ ciphertext stealing cho độ dài không chia hết 16 (cần ≥ 16 byte).
- **Không có tính toàn vẹn**: kẻ tấn công vẫn có thể hoán vị/thay thế cả khối 16 byte (hỏng tại chỗ). Chỉ phù hợp bảo mật dữ liệu *at rest* khi có lớp toàn vẹn khác.

2.8 CCM — Counter with CBC-MAC (AEAD)

- Kết hợp **CTR** để mã hóa và **CBC-MAC** để tính tag trên (nonce, AAD, plaintext).
- Chuẩn: NIST SP 800-38C. Tham số: độ dài nonce $n \in [7, 13]$ byte, độ dài tag $t \in \{4, 6, \ldots, 16\}$ byte; trường độ dài thông điệp $q = 15 - n$ byte ⇒ thông điệp tối đa 2^{8q} byte (với nonce 12 B ⇒ $q = 3$ ⇒ ~16 MB, đúng như ghi trong README).
- **Hai lượt (two-pass)** và phần CBC-MAC tuần tự ⇒ chậm hơn GCM (mục 6.5). Cần biết trước độ dài dữ liệu (`SpecifyDataLengths`), nên không stream được thuần túy.

2.9 GCM — Galois/Counter Mode (AEAD)

- Mã hóa bằng **CTR**; tag tính bằng **GHASH** — đa thức trên GF(2^{128}) với khóa $H = E_K(0^{128})$:

$$\text{GHASH}_H(A, C) = \sum (\text{các khối A, C, độ dài}) \cdot H^i, \qquad T = E_K(J_0) \oplus \text{GHASH}_H(A, C)$$

- Nonce chuẩn **96 bit** ($J_0 = \text{Nonce} \,\|\, 0^{31} \,\|\, 1$). Nhờ lệnh **PCLMULQDQ** và AES-NI, GCM đạt thông lượng gần CTR.

- **Bắt buộc nonce duy nhất cho mỗi khóa**: tái sử dụng nonce làm lộ $P_1 \oplus P_2$ **và** cho phép khôi phục khóa xác thực $H \Rightarrow$ **giả mạo tag tùy ý** (forgery). Với nonce ngẫu nhiên 96 bit, giới hạn an toàn thực tế ~2^{32} thông điệp/khóa (va chạm sinh nhật).

2.10 Bảng so sánh tổng hợp

Tiêu chí	ECB	CBC	OFB	CFB	CTR	XTS	CCM	GCM
Cần IV/nonce	✗	✓ ngẫu nhiên	✓ duy nhất	✓ ngẫu nhiên	✓ duy nhất	✓ tweak	✓ duy nhất	✓ duy nhất
Cần padding	✓	✓	✗	✗	✗	✗ (CTS)	✗	✗
Mã hóa song song	✓	✗	✗	✗	✓	✓	✗ (MAC)	✓
Giải mã song song	✓	✓	✗	✓	✓	✓	✗ (MAC)	✓
Bảo mật (confidentiality)	✗ (lộ mẫu)	✓	✓	✓	✓	✓ (tĩnh)	✓	✓
Toàn vẹn / xác thực	✗	✗	✗	✗	✗	✗	✓	✓
Rủi ro đặc trưng	Lộ mẫu	Padding oracle	Reuse IV	–	Reuse nonce	Không toàn vẹn	Reuse nonce	Reuse nonce (mất cả tag)

3. Thiết kế công cụ CLI & cơ chế chống lạm dụng

3.1 Kiến trúc mã nguồn

```
src/aescore.{h,cpp}      Lõi bọc Crypto++: mode, kiểm tra tham số, CSPRNG, key id,
secure_wipe
src/container.{h,cpp}    Định dạng container "AEST" + sidecar JSON
src/ops.{h,cpp}          Pipeline encrypt/decrypt + chính sách ECB + phát hiện
nonce reuse
src/kat.{h,cpp}          KAT runner (--kat)            src/minijson.h   Parser
JSON tối giản
src/bench.{h,cpp}        Benchmark + thống kê          src/util.{h,cpp}
hex/base64/UTF-8/file I/O
src/aescore_capi.*       C API cho .dll/.so            src/main.cpp     CLI
(aestool)
tests/test_aescore.cpp   13 test case Catch2           vectors/nist_vectors.json
vector NIST
aestool_gui.py           GUI Python (tkinter + ctypes → aescore)
```

Luồng dữ liệu: `main.cpp` (phân tích CLI) → `ops.cpp` (chính sách + phát hiện nonce reuse) → `aescore.cpp` (Crypto++) → `container.cpp` (đóng gói) → ghi file **nguyên tử** (ghi `*.aestool.tmp` rồi đổi tên, không bao giờ để lại file hỏng).

Phân cấp lỗi (đều dẫn xuất từ `CryptoError` để mọi nhánh lỗi đều "fail closed"):

Lớp lỗi	Ý nghĩa	Mã thoát CLI
`UsageError`	Từ chối dùng sai (ECB quá lớn, nonce reuse, độ dài IV/khóa sai…)	**2**
`AuthError`	Tag không khớp	**3**
`FormatError`	Container/sidecar sai định dạng	**3**
`CryptoError`	Lỗi mật mã khác (padding sai…)	**3**
Lỗi CLI chung / KAT fail	Cú pháp, KAT thất bại	**1**

3.2 Cú pháp lệnh `aestool`

```
aestool encrypt --mode MODE (--key FILE | --key-hex HEX) (--in FILE | --text STR)
[--out FILE]
                [--iv FILE|HEX | --nonce FILE|HEX] [--aad FILE | --aad-text STR]
[--aead]
                [--tag-len N] [--encode hex|base64|raw] [--sidecar FILE.json] [--
ledger FILE]
                [--allow-ecb] [--verbose]
aestool decrypt [--mode MODE] (--key FILE | --key-hex HEX) (--in FILE | --text
ENCODED) [--out FILE]
                [--iv ..] [--nonce ..] [--aad ..] [--encode hex|base64] [--sidecar
FILE.json] [--allow-ecb]
aestool keygen  [--bits 128|192|256|512] [--format raw|hex] [--out FILE] [--force]
aestool --kat vectors.json [--verbose]
aestool bench   [--modes gcm,ctr,...] [--sizes 1k,4k,16k,256k,1m,8m] [--runs 1000]
[--warmup 1.0]
                [--bits 256] [--dir enc|dec|both] [--with-io] [--csv FILE] [--
quick]
aestool version | help
```

Nhập khóa: `--key FILE` (khuyến nghị) hoặc `--key-hex HEX` (chương trình in `NOTE` cảnh báo khóa lộ trong shell history và danh sách tiến trình). **Mã hóa đầu ra (`--encode`):** `hex` (mặc định), `base64`, `raw` (nhị phân, bắt buộc có `--out`, không bao giờ in dữ liệu nhị phân ra terminal). **Văn bản UTF-8:** `--text "xin chào"` mã hóa chuỗi UTF-8; file nhị phân bất kỳ dùng `--in`.

3.3 Các ví dụ thực thi (kết quả thật)

(a) Sinh khóa AES-256 — file tạo với quyền `600` trên Linux, không ghi đè file có sẵn (trừ khi `--force`):

```
$ aestool keygen --bits 256 --out key.bin
Key (256 bits, raw) written to key.bin
  key id: e22ed6bdc1c331b9
  Protect this file (Linux: chmod 600 applied; Windows: restrict the folder ACL).
[exit=0]
$ ls -l key.bin
-rw------- 1 user user 32 key.bin
```

(b) Mã hóa AES-256-GCM có AAD, IV tự sinh lưu trong header:

```
$ aestool encrypt --mode gcm --key key.bin --in msg.txt --out ct_gcm.bin --aad-
text "v1" --verbose
Encrypted 57 bytes with AES-256-GCM -> ct_gcm.bin
  IV/nonce : 08f3196938704dfd6a9fde16  (random, stored in file header)
  tag      : d2e93b850e353e95de342c40740e71c8
  key id   : e22ed6bdc1c331b9
  ciphertext (hex): 4145535401082001e22ed6bd...      (49 45 53 54 = "AEST")

$ aestool decrypt --key key.bin --in ct_gcm.bin --out dec.txt
Decrypted 57 bytes (AES-256-GCM)
  written to dec.txt
$ cat dec.txt
Xin chào, đây là bản tin mật số 1 của Lab 1.       # round-trip OK (cmp dec.txt
msg.txt → giống nhau)
```

(c) Mã hóa chuỗi UTF-8 ra Base64:

```
$ aestool encrypt --mode gcm --key key.bin --text "xin chào" --aad-text "v1" --
encode base64
QUVTVAEIIAHiLta9wcMxuQzAxyCIng289sGK3awAAAACdjEQCmtIM3sEz/PYAF1hcbx6NgAAAAAAAAAJhJ
iGq2rXf01N
```

(d) Sidecar JSON — file đầu ra chỉ chứa ciphertext thô, metadata nằm trong JSON:

```
$ aestool encrypt --mode ctr --key key.bin --in msg.txt --out ct_ctr.bin --sidecar
ct_ctr.json
Encrypted 57 bytes with AES-256-CTR -> ct_ctr.bin
  IV/nonce : 904597fe17a4180923e507bca968f2bb  (random, stored in sidecar)
  sidecar  : ct_ctr.json
$ cat ct_ctr.json
{
  "format": "aestool-sidecar-v1",
  "alg": "AES-256-CTR",
  "mode": "ctr",
  "key_len": 32,
  "keyid": "e22ed6bdc1c331b9",
```

```
  "iv": "904597fe17a4180923e507bca968f2bb",
  "aad": "",
  "tag": ""
}
$ aestool decrypt --key key.bin --in ct_ctr.bin --sidecar ct_ctr.json --out
dec_ctr.txt
Decrypted 57 bytes (AES-256-CTR)
```

(e) CCM và XTS:

```
$ aestool encrypt --mode ccm --key key.bin --in msg.txt --out ct_ccm.bin --aad-
text meta
Encrypted 57 bytes with AES-256-CCM -> ct_ccm.bin
  IV/nonce : 3f8592d8daf5fb51f7e334df  (random, stored in file header)
  tag      : 0d139134815f38e86302bb89da5654a3
$ aestool encrypt --mode xts --key-hex <128 ký tự hex = 64 byte> --in msg.txt --
out ct_xts.bin
Encrypted 57 bytes with AES-256-XTS -> ct_xts.bin        # 57 B không chia hết 16
→ ciphertext stealing
```

3.4 Định dạng container nhị phân

Tất cả trường nhiều byte theo **big-endian**:

Offset	Trường	Kích thước	Ý nghĩa
0	`magic`	4	`"AEST"`
4	`version`	1	`1`
5	`mode`	1	1=ECB … 8=GCM
6	`key_len`	1	16/24/32 (XTS: 32/64)
7	`flags`	1	bit0 = AEAD
8	`key_id`	8	8 byte đầu của `SHA-256("aestool-keyid-v1" ‖ key)`
16	`iv_len`	1	
17	`iv`	`iv_len`	
…	`aad_len`	4	
…	`aad`	`aad_len`	
…	`tag_len`	1	
…	`tag`	`tag_len`	
…	`ct_len`	8	
…	`ciphertext`	`ct_len`	

Ví dụ GCM ở (b): `magic(4) + ver(1) + mode(1)=08 + keylen(1)=20(32) + flags(1)=01 + keyid(8) + ivlen(1)=0c + iv(12) + aadlen(4)=00000002 + aad("v1") + taglen(1)=10 + tag(16) + ctlen(8)=0x39(57) + ct(57)` → tổng **117 byte** (khớp kích thước file khi chạy thật).

Bộ phân tích (parser) kiểm tra chặt: magic, version, mode hợp lệ, cờ AEAD khớp mode, độ dài khóa/IV/tag hợp lệ cho mode, **độ dài ciphertext phải khớp đúng kích thước file** — sai một điều kiện là từ chối (không bao giờ đoán).

> `key_id` cho phép phát hiện tái sử dụng nonce theo *cặp (khóa, nonce)* mà **không lộ khóa** (băm có domain-separation, chỉ giữ 64 bit).

3.5 Cơ chế kiểm tra chống lạm dụng (Misuse Checks)

3.5.1 Hạn chế ECB

- Chọn ECB luôn in `WARNING` (stderr):

```
$ aestool encrypt --mode ecb --key key.bin --in msg.txt --out ct_ecb.bin
WARNING: ECB mode is NOT secure: identical plaintext blocks produce identical
ciphertext blocks and there is no integrity protection. Prefer GCM or CCM.
Encrypted 57 bytes with AES-256-ECB -> ct_ecb.bin
```

- Dữ liệu **> 16 KiB** bị từ chối (mã thoát 2) trừ khi có `--allow-ecb`:

```
$ aestool encrypt --mode ecb --key key.bin --in big.bin --out ct_ecb_big.bin
# big.bin = 20000 B
ERROR: ECB refused: data is 20000 bytes, above the default 16 KiB limit. Use
--allow-ecb only if you really must.
[exit=2]
$ aestool encrypt --mode ecb --key key.bin --in big.bin --out ct_ecb_big.bin
--allow-ecb
Encrypted 20000 bytes with AES-256-ECB -> ct_ecb_big.bin      [exit=0]
```

3.5.2 Quản lý IV / Nonce

Mode	Độ dài IV/nonce bắt buộc	Kiểm tra
CBC, OFB, CFB, CTR, XTS	16 B	`valid_iv_len`
GCM	12 B (96 bit)	chế độ thường: đúng 12 B
CCM	7–13 B (mặc định 12)	
ECB	không có IV	cung cấp IV → từ chối

- Thiếu IV ⇒ sinh bằng `CryptoPP::AutoSeededRandomPool` (hàm `random_bytes`), lưu trong header hoặc sidecar. Lỗi RNG hệ thống ⇒ `CryptoError` (fail closed).
- Sai độ dài ⇒ từ chối **ngay ở khâu kiểm tra tham số**, trước khi gọi Crypto++:

```
$ aestool encrypt --mode gcm --key key.bin --in msg.txt --out x.bin --nonce-
hex 0011223344
ERROR: invalid IV/nonce length 5 bytes for GCM (expected 12 bytes)
[exit=2]
```

3.5.3 Chống tái sử dụng nonce (CTR, CCM, GCM)

Hàm `nonce_reused` (ops.cpp) chạy **trước** khi gọi Crypto++, theo thứ tự:

```
1. kid = key_id(key)                      // SHA-256 có domain-separation, 8 byte
2. Nếu --out đã tồn tại và là file AEST  → đọc CHỈ header (≤ 1 MiB), so sánh
(key_id, iv)
3. Nếu --sidecar đã tồn tại              → parse JSON, so sánh (keyid, iv)
4. Nếu có --ledger FILE                   → mỗi dòng "<keyid-hex> <nonce-hex>";
dòng trùng → từ chối
5. Nonce do người dùng đưa vào (--nonce) mà trùng → UsageError "NONCE REUSE
REFUSED" (mã 2)
6. Nonce tự sinh mà trùng (cực hiếm)    → sinh lại, tối đa 8 lần, rồi CryptoError
7. Sau khi mã hóa thành công, ghi (kid, nonce) vào ledger (nonce đã dùng thì coi
là "cháy")
```

Kết quả chạy thật:

```
$ aestool encrypt --mode gcm --key key.bin --in msg.txt --out r1.bin --nonce-hex
000102030405060708090a0b
Encrypted 57 bytes with AES-256-GCM -> r1.bin        # lần 1: chấp nhận
$ aestool encrypt --mode gcm --key key.bin --in msg.txt --out r1.bin --nonce-hex
000102030405060708090a0b
ERROR: NONCE REUSE REFUSED: same key + nonce found in the header of existing
output file 'r1.bin'.
       Reusing a nonce with the same key breaks GCM. Omit --nonce to generate a
fresh random one.   [exit=2]

$ aestool encrypt ... --out r2.bin --nonce-hex 000102030405060708090a0b --ledger
led.txt   # ghi vào sổ cái
$ aestool encrypt ... --out r3.bin --nonce-hex 000102030405060708090a0b --ledger
led.txt
ERROR: NONCE REUSE REFUSED: same key + nonce recorded in ledger 'led.txt'. ...
[exit=2]
$ cat led.txt
e22ed6bdc1c331b9 000102030405060708090a0b
```

Giới hạn (trung thực): phát hiện chỉ dựa trên file/ledger mà chương trình *nhìn thấy*; không bắt được tái sử dụng giữa nhiều máy hoặc khi header đã bị xóa. Với CTR chỉ phát hiện nonce **trùng hoàn toàn**, không phát hiện các dải counter chồng lấn. Giải pháp triệt để trong hệ thống thật là nonce ngẫu nhiên 96 bit + đổi khóa trước $\sim 2^{32}$ thông điệp, hoặc bộ đếm có trạng thái bền vững.

3.5.4 Vệ sinh bộ nhớ & I/O

- Khóa được giữ trong `KeyGuard` (RAII) và **ghi đè 0 bằng con trỏ `volatile`** (`secure_wipe`) khi ra khỏi phạm vi, kể cả khi có ngoại lệ.
- `keygen` không ghi đè file có sẵn; `chmod 600` trên Linux.
- Ghi file nguyên tử (`*.aestool.tmp` → rename).
- Giới hạn 2 GiB/file đầu vào; sidecar/ledger đọc có giới hạn kích thước.